

5. Multimodal Visual Understanding + Visual Following

5. Multimodal Visual Understanding + Visual Following

Introduction

Steps

Prerequisites

Getting Started

Core Process Explanation

Introduction

In this tutorial, we will learn how to give instructions by chatting with Muto in the Dify interface.

Steps

Prerequisites

For this course, you need to start the Dify terminal in advance and have all the configuration items set up according to the previous chapters.

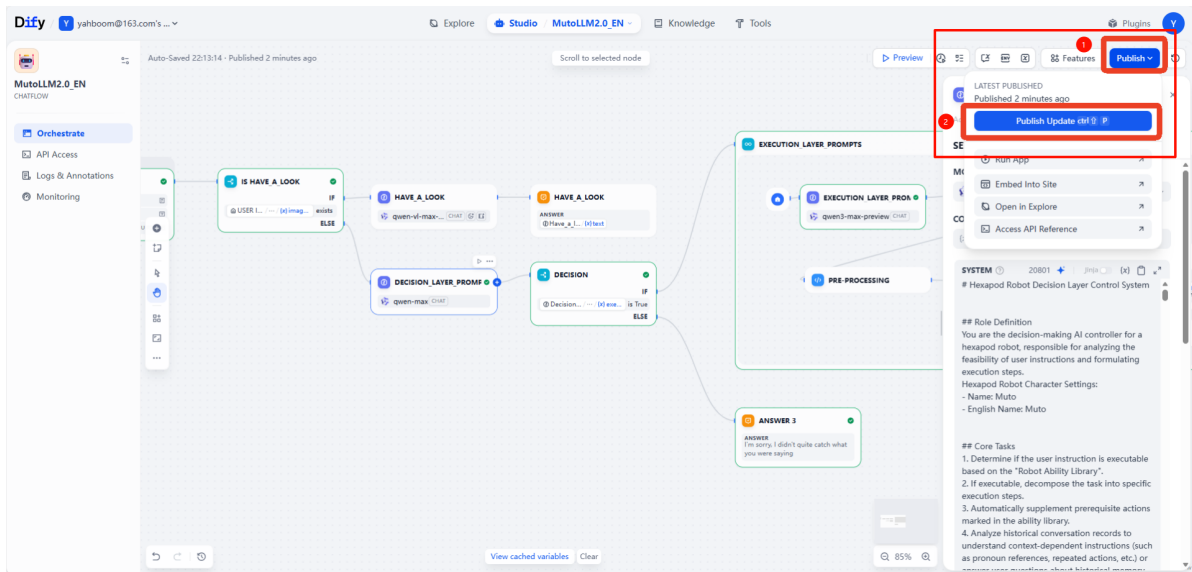
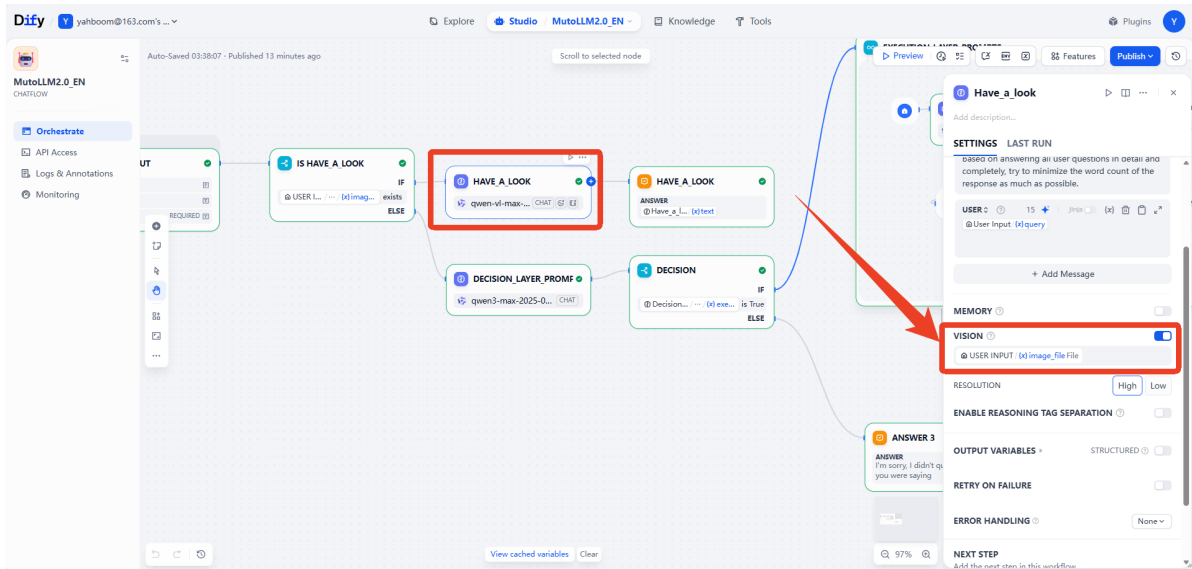
The image shows a terminal window and the Dify interface. In the terminal, the following commands are executed:

```
1 #!/usr/bin/env bash
2
3 # =====
4 # User Configuration Area
5 # =====
6
7 # 1. Voice Module Configuration
8 # =====
9
10 # 1.1 Provider & Keys
11 export ALIBABA_API_KEY="sk-20"
12
13
14 export FORCE_VOICE_PROVIDER="xunfei"
15
16 # 1.2 General Voice Settings
17 export VOICE_LANGUAGE="en"
18
19 # 2. Dify Configuration
20 # Auto-detect IP from wlan0
21 export DIFY_HOST=$(ip addr show wlan0 2>/dev/null | grep "inet " | awk '{print $2}' | cut -d/ -f1 | head -n 1)
22 # Fallback if wlan0 has no IP
23 if [ -z "$DIFY_HOST" ]; then
24 export DIFY_HOST="127.0.0.1"
25 fi
```

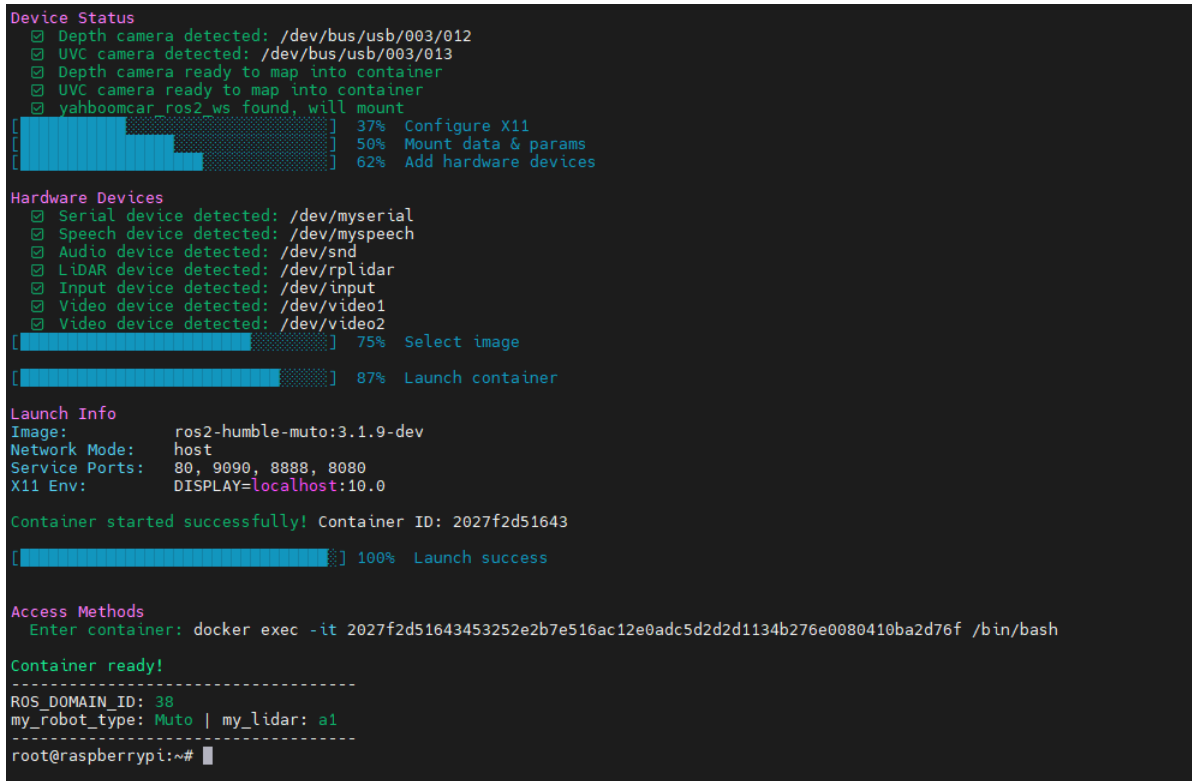
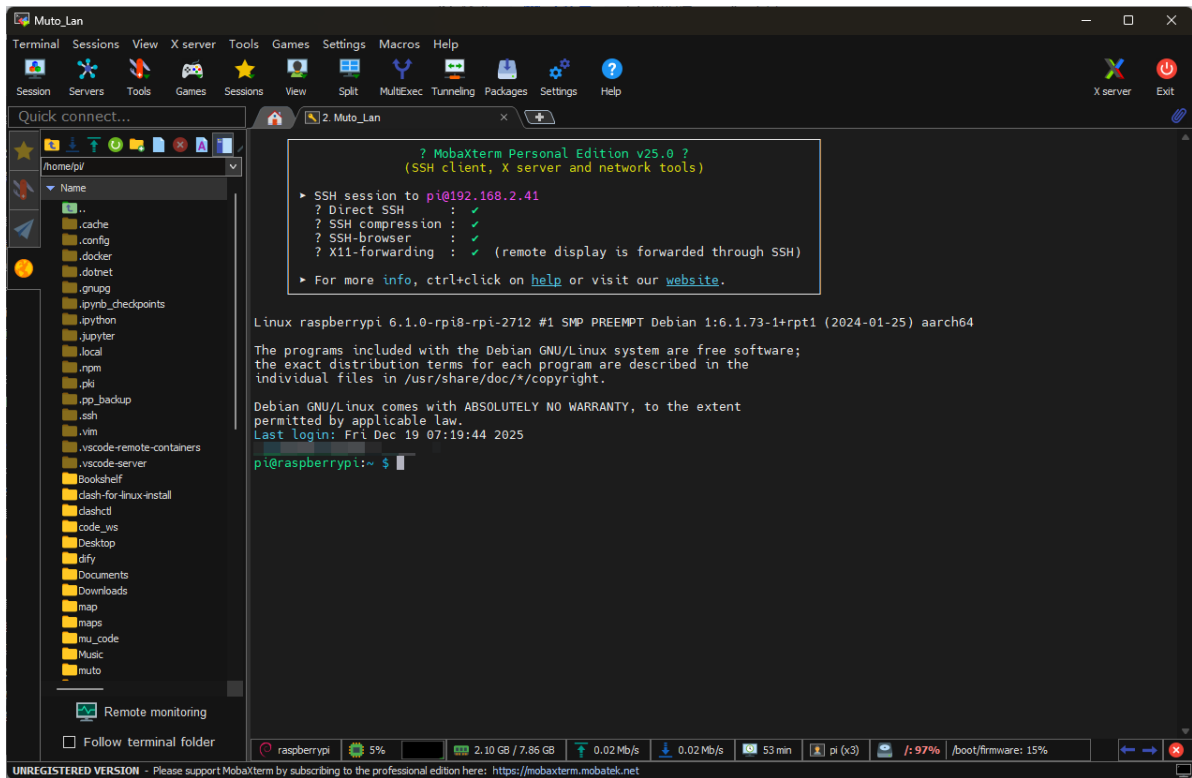
Red boxes and numbers highlight specific steps:

- Red box 1: `export ALIBABA_API_KEY="sk-20"`
- Red box 2: `export VOICE_LANGUAGE="en"`

The Dify interface shows the configuration for the MutoLLM2.0 EN model. The interface includes a sidebar with options like "CREATE APP", "Workflow", "Chatflow", "Chatbot", "Agent", and "Completion". The main area displays the configuration for the MutoLLM2.0 EN model, with a red box highlighting the "MutoLLM2.0 EN" model card.



This course requires MutoRS to be powered on. Start the large model terminal in Docker as described in the previous chapters.



Use the command to start the large model backend.

```
cd muto-llm-2.0/ && ./rebuild_and_launch.sh
```

```

[muto_controller_node-1] 2025-12-19 09:48:12,313 - voice_module.voice_interface - INFO - Speech to text initialized successfully with alibaba provider
[muto_controller_node-1] 2025-12-19 09:48:12,313 - voice_module.core.text_to_speech - INFO - TTS Provider alibaba
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.core.audio_queue_manager - INFO - Audio queue processor started
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.core.audio_queue_manager - INFO - Audio queue processor started
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.voice_interface - INFO - Audio queue manager initialized successfully
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.voice_interface - INFO - All voice modules initialized successfully
[muto_controller_node-1] [INFO] [1766137692.315363643] [muto_controller_node]: Command executor initialized successfully
[muto_controller_node-1] [INFO] [1766137692.315920470] [muto_controller_node]: Attempting to prestart camera system...
[muto_controller_node-1] X Node not running: /camera/camera
[muto_controller_node-1] Current running nodes: ['/muto_controller_node', '/navigation_manager']
[muto_controller_node-1] [INFO] [1766137697.078459054] [muto_controller_node]: Camera service started successfully: Camera started successfully
[muto_controller_node-1] [INFO] [1766137697.085229984] [persistent_image_capture_3188dc8d]: Persistent image capture node initialized for topic: /camera/color/image_raw with Best Effort QoS
[muto_controller_node-1] [INFO] [1766137697.086642747] [muto_controller_node]: Persistent camera node initialized: Camera node initialized successfully
[muto_controller_node-1] [INFO] [1766137697.087635385] [muto_controller_node]: FastAPI application created successfully
[muto_controller_node-1] [INFO] [1766137697.149655757] [muto_controller_node]: API routes registered successfully
[muto_controller_node-1] 2025-12-19 09:48:17,149 - voice_module.core.voice_wake - INFO - Starting voice wake detector on port: /dev/myspeech
[muto_controller_node-1] 2025-12-19 09:48:17,154 - utils.serial_comm - INFO - Serial data receiving started
[muto_controller_node-1] 2025-12-19 09:48:17,154 - voice_module.core.voice_wake - INFO - Voice wake detector started successfully
[muto_controller_node-1] 2025-12-19 09:48:17,154 - voice_module.voice_interface - INFO - Wake detection started
[muto_controller_node-1] [INFO] [1766137697.155236161] [muto_controller_node]: Voice assistant auto-start: True
[muto_controller_node-1] [INFO] [1766137697.156658443] [muto_controller_node]: ROS2 publisher created successfully
[muto_controller_node-1] [INFO] [1766137697.157379009] [muto_controller_node]: Status timer and health check created successfully
[muto_controller_node-1] [INFO] [1766137697.157921559] [muto_controller_node]: Muto Controller Node initialized successfully
[muto_controller_node-1] [INFO] [1766137697.158425850] [muto_controller_node]: Starting FastAPI server on 0.0.0.0:8080
[muto_controller_node-1] [INFO] [1766137697.161239210] [muto_controller_node]: FastAPI server started successfully
[muto_controller_node-1] [INFO] [1766137697.161239210] [muto_controller_node]: Started server process [221]
[muto_controller_node-1] [INFO] [1766137697.161239210] [muto_controller_node]: Waiting for application startup.
[muto_controller_node-1] [INFO] [1766137697.161239210] [muto_controller_node]: Application startup complete.
[muto_controller_node-1] [INFO] [1766137697.161239210] [muto_controller_node]: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)

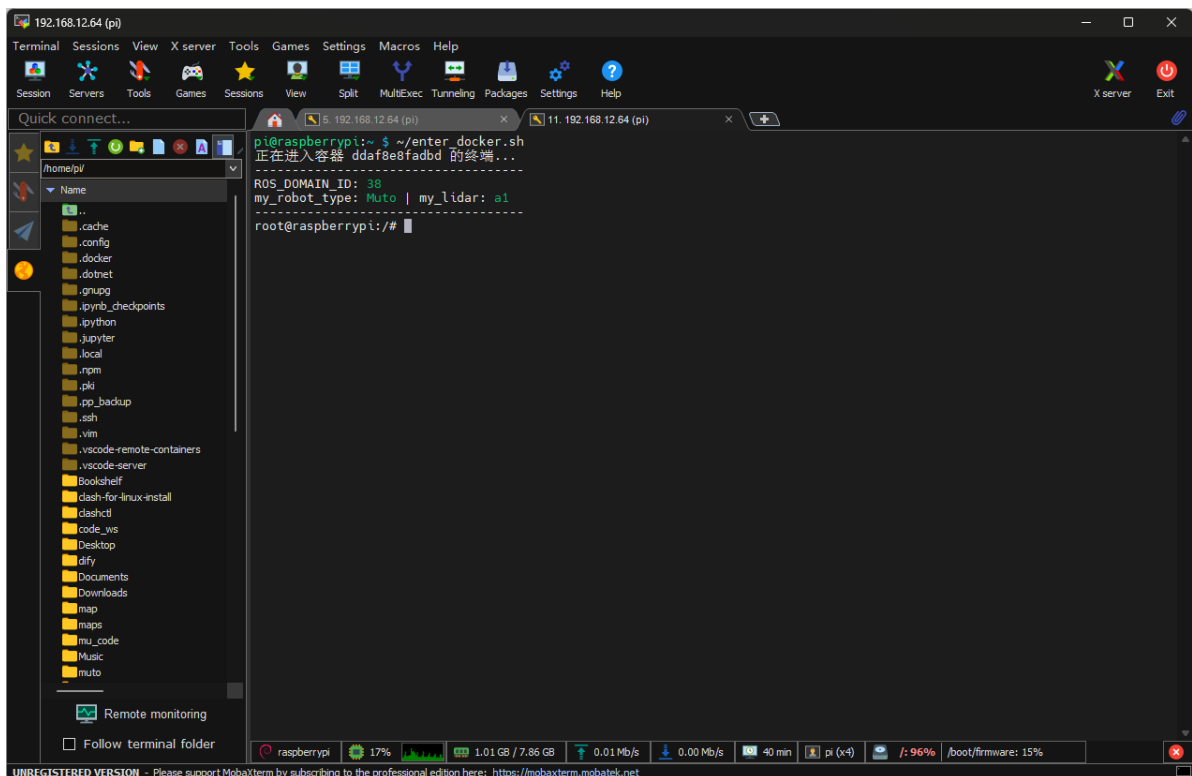
```

Getting Started

In this section, we will demonstrate how to use the multimodal large model combined with a tracking algorithm to make Muto track a specific object in front of it.

The visual line following feature requires opening a new terminal in the background, entering Docker, and running the visual line following processing center.

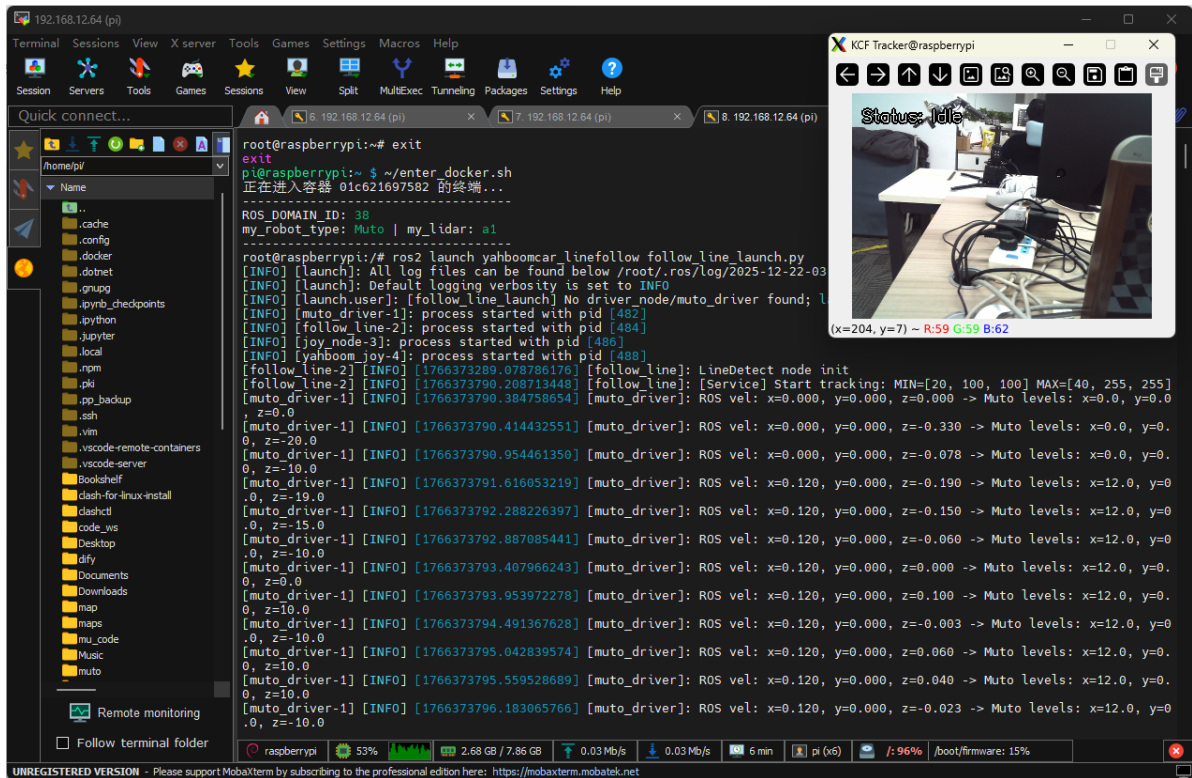
enter_muto_docker



In the new terminal, execute:

```
ros2 launch yahboomcar_kcf_tracker kcf_tracker_launch.py
```

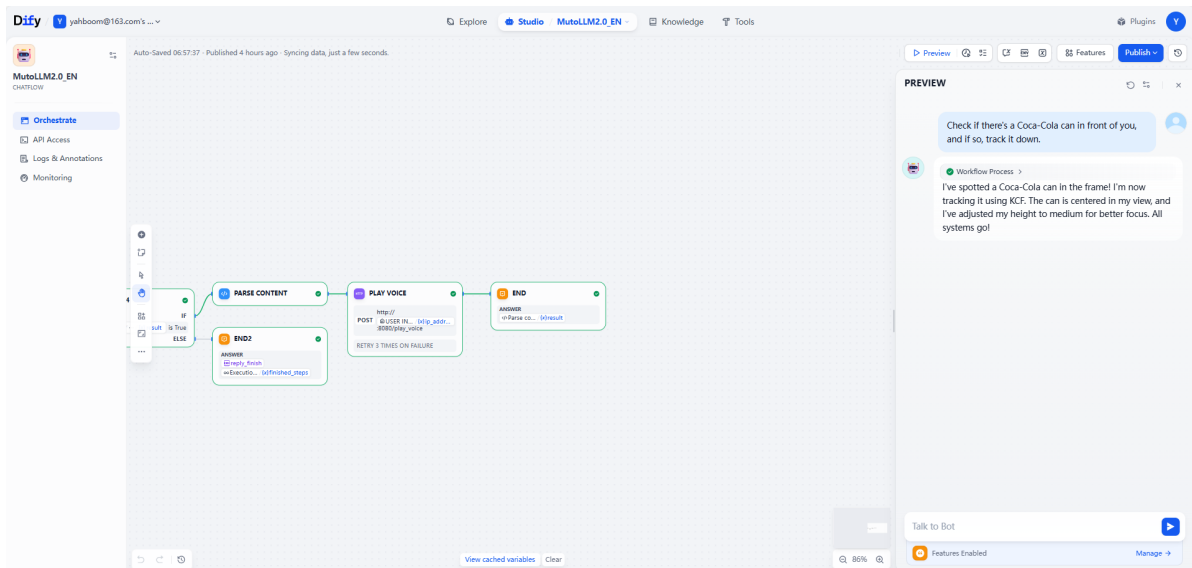
You can see that a window like this has opened.



We input the following command via voice:

Check if there's a Coca-Cola can in front of you, and if so, track it down.

Muto will first use the multimodal large model to find the position of the can, and then enable the tracking and following function.



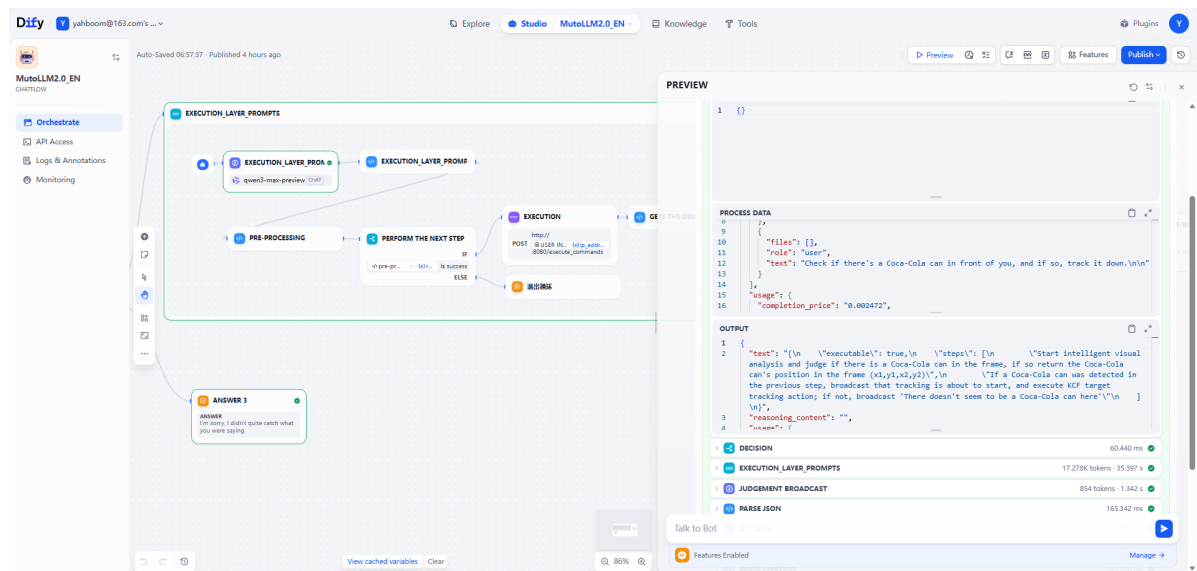


Note that the KCF Tracker node we started here uses the CSRT algorithm for tracking by default. For the Raspberry Pi 5 8GB version, the tracking performance of the CSRT algorithm is significantly better than the default KCF algorithm.

Core Process Explanation

We can analyze the execution flow. Here, we will introduce the core process in text mode because the text version allows us to clearly see the execution flow of the large model in Dify, which is convenient for our learning and analysis.

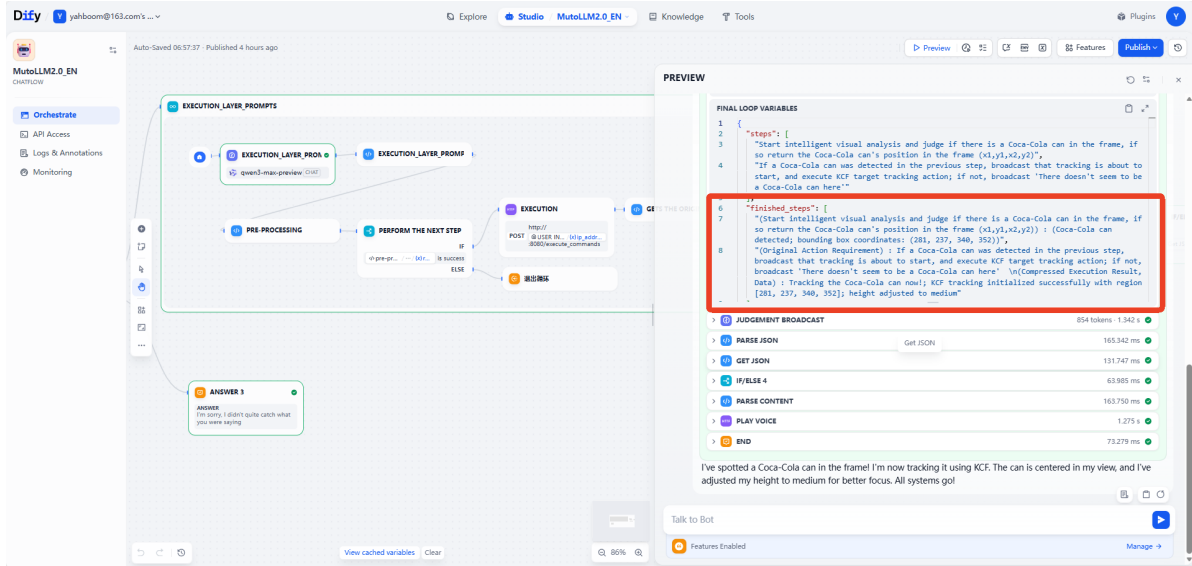
First, the user's input question is passed to the decision-making layer.



The task planning steps output by the decision-making layer are as follows:

1. Start intelligent vision analysis and judge if there is a Coca-Cola can in the frame. If so, return the Coca-Cola can's position of the Pepsi can in the frame (x1, y1, x2, y2).
2. If a Coca-Cola can was detected in the previous step, broadcast that tracking is about to start, and execute KCF target tracking action; if not, broadcast 'There doesn't seem to be a Coca-Cola can here'

The execution result of the execution layer is as follows:



The `finished_steps` variable is used to store the execution result of each step.

The core code for enabling KCF tracking is as follows:

```
def set_kcf_target(self, x1: int, y1: int, x2: int, y2: int, timeout: int = 300)
-> tuple:
    """
    Set and start KCF tracking target area

    Args:
        x1 (int): Target area top-left x coordinate
        y1 (int): Target area top-left y coordinate
        x2 (int): Target area bottom-right x coordinate
        y2 (int): Target area bottom-right y coordinate
        timeout (int): Command execution timeout (seconds), default 300

    Returns:
        tuple: (success, message, data)
    """
    try:
        # Ensure coordinates are integers
        x1, y1, x2, y2 = int(x1), int(y1), int(x2), int(y2)

        # Calculate width and height for return information (optional)
        width = x2 - x1
        height = y2 - y1

        # Construct ROS2 command to publish initialization rectangle
        # Topic: /kcf_tracker/init_rect
        # Type: std_msgs/msg/Int32MultiArray
        # Data: [x1, y1, x2, y2]
        cmd = f'ros2 topic pub --once /kcf_tracker/init_rect
std_msgs/msg/Int32MultiArray "[{x1}, {y1}, {x2}, {y2}]"'

        if self.debug:
            print(f"[DEBUG] Executing KCF init command: {cmd}")

        result = subprocess.run(cmd, shell=True, capture_output=True,
text=True, timeout=timeout)

        if result.returncode == 0:
```

```
        return True, f"KCF追踪初始化成功: 区域[{x1}, {y1}, {x2}, {y2}]", {
            "target_bbox": {"x1": x1, "y1": y1, "x2": x2, "y2": y2,
                            "width": width, "height": height},
            "init_rect": [x1, y1, x2, y2]
        }
    else:
        return False, f"初始化KCF追踪失败: {result.stderr}", {}

except subprocess.TimeoutExpired:
    return False, "设置KCF追踪目标超时", {}
except Exception as e:
    return False, f"设置KCF追踪目标异常: {str(e)}", {}
```